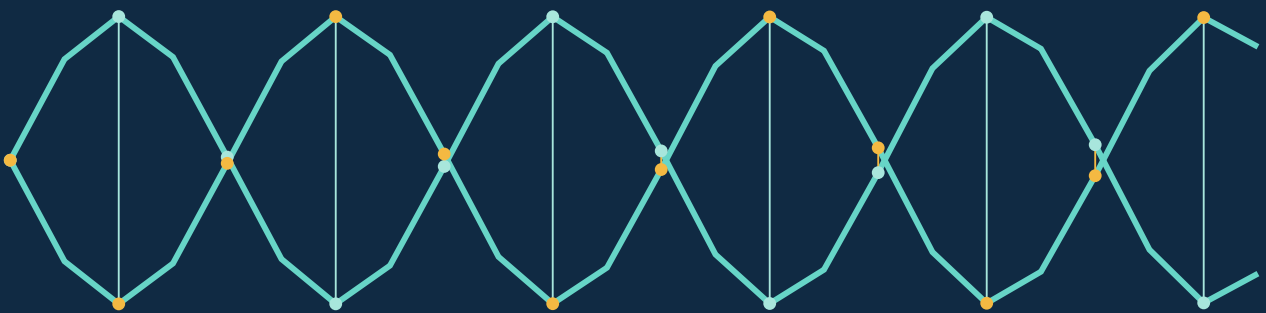


GUIDE UTILISATEUR

BioPHP

Parser, explorer et analyser des données biologiques en PHP



Deux exemples fil rouge

SwissProt - protéine TNF humaine (P01375)

Genome - statistiques de séquençage de Homo sapiens

Édition du 25 août 2026 | Snapshot v1.0.0-2-g45feef8

À propos de ce guide

Ce document part de deux fichiers réels livrés avec la bibliothèque - `data/basicswiss.txt` et `data/sample.genome` - puis élargit le parcours à l'ensemble de l'API publique utile : 19 parseurs, séquences typées, analyses, alignements, comparaison, protéines, enzymes de restriction, oligonucléotides, outils et adaptateurs REST.

Périmètre exact

Documentation fondée sur le code et les tests du dépôt au commit 45feef8. La bibliothèque est annoncée en version alpha : vérifiez les notes de version avant une montée de version.

Sommaire

À propos de ce guide	2
1. Démarrage rapide	3
2. Exemple complet - SwissProt	5
3. Exemple complet - Genome	7
4. Lire directement ou indexer avec Doctrine	9
5. Catalogue des 19 parseurs	10
6. Séquences sûres avec les value objects	11
7. Calculs sur les séquences	12
8. Alignements et comparaison	13
9. Protéines et enzymes de restriction	15
10. Oligonucléotides, génétique et statistiques	16
11. Adaptateurs REST et DTO	17
12. Intégration Symfony	18
13. Recettes de bout en bout	19
14. Erreurs, limites et bonnes pratiques	21
Annexe A. Référence rapide	22

1. Démarrage rapide

BioPHP est à la fois une bibliothèque PHP 8.2+ et un bundle Symfony 7. Le chemin le plus court pour lire un enregistrement ne requiert ni base Doctrine ni appel réseau : chargez le fichier en mémoire et passez ses lignes à la factory de lecture.

Installation

```
composer require amelaye/biophp
```

Deux familles de fonctionnalités

Les parseurs et value objects fonctionnent localement. Plusieurs calculs historiques (poids moléculaire, traduction, enzymes) reçoivent des adaptateurs vers l'API de données biologiques.

Lire SwissProt en 12 lignes

```
<?php
require 'vendor/autoload.php';

use Amelaye\BioPHP\Domain\Database\Factory\DatabaseReaderFactory;

$parser = DatabaseReaderFactory::readDatabase(
    'SWISSPROT',
    file(__DIR__ . '/data/basicswiss.txt')
);

$sequence = $parser->getSequence();
printf("%s - %d aa\n", $sequence->getPrimAcc(), $sequence->getSeqLength());
printf("%s\n", $sequence->getDescription());
```

Lire Genome en 10 lignes

```
<?php
require 'vendor/autoload.php';

use Amelaye\BioPHP\Domain\Database\Factory\DatabaseReaderFactory;

$genome = DatabaseReaderFactory::readDatabase(
    'GENOME',
    file(__DIR__ . '/data/sample.genome')
);

printf("%s : %s pb\n", $genome->getOrganism(), $genome->getSize());
```

Le modèle mental



La factory choisit un parseur par nom de format. Le parseur conserve l'état du record lu et expose ensuite ses données par getters. Les résultats SwissProt, GenBank et EMBL utilisent le modèle commun Sequence / Feature / Reference ; Genome expose des scalaires et des GenomeReference.

Votre situation	Point d'entrée
Un fichier, résultat immédiat	DatabaseReaderFactory
Une collection consultée par identifiant	DatabaseManager + Doctrine
Une chaîne à valider / transformer	Value objects
Des calculs dépendant des référentiels	Managers + adaptateurs API

2. Exemple complet - SwissProt

Le fixture décrit le précurseur du facteur de nécrose tumorale humain. Le parseur lit les lignes ID, AC, DT, DE, GN, OS, OC, RN/RA/RL/RX, KW, FT, DR et SQ, puis les répartit dans les entités du modèle commun.

P01375 accession primaire	233 aa longueur	30 features	10 références
-------------------------------------	---------------------------	-----------------------	-------------------------

Parser et inspecter toutes les familles de résultats

```
use Amelaye\BioPHP\Domain\Database\Factory\DatabaseReaderFactory;

$swiss = DatabaseReaderFactory::readDatabase('SWISSPROT', file('data/basicswiss.
txt'));
$seq = $swiss->getSequence();

echo $seq->getPrimAcc(); // P01375
echo $seq->getEntryName(); // FA (valeur actuelle du parseur)
echo $seq->getMolType(); // PRT;
echo $seq->getSeqLength(); // 233
echo $seq->getSource(); // HOMO SAPIENS (HUMAN)
echo $seq->getDescription();
echo substr($seq->getSequence(), 0, 30);

$secondary = $swiss->getAccession();
$authors = $swiss->getAuthors();
$features = $swiss->getFeatures();
$keywords = $swiss->getKeywords();
$refs = $swiss->getReferences();
$xrefs = $swiss->getSpDatabank();
```

Objets renvoyés

Getter du parseur	Type	Informations principales
getSequence()	Sequence	accession, nom, longueur, type, date, source, description, taxonomie, chaîne
getAccession()	Accession[]	accessions secondaires rattachées à l'accension primaire
getAuthors()	Author[]	auteur et numéro de référence
getFeatures()	Feature[]	clé, intervalle, qualificateur, valeur, description
getKeywords()	Keyword[]	mots-clés du record
getReferences()	Reference[]	titre, journal, PubMed/Medline, remarques
getSpDatabank()	SpDatabank[]	références croisées DR : banque, identifiants
getGbSequence() / getSrcForm()	GbSequence / SrcForm	objets communs présents dans la base abstraite ; surtout pertinents pour GenBank/EMBL

Passer à une séquence protéique typée

```
use Amelaye\BioPHP\Domain\Sequence\ValueObject\MolecularSequenceFactory;

$protein = MolecularSequenceFactory::fromEntity($seq);

echo $protein->getMolType();      // PROTEIN
echo $protein->getLength();       // 233
echo $protein->countSymbol('C');
echo $protein->subSequence(0, 20);
var_dump($protein->hasStop());
var_dump($protein->hasUnknownResidue());
```

Important

Les positions retournées par les fonctions de motifs et subSequence() sont indexées à partir de 0. Les coordonnées biologiques des features peuvent suivre la convention du format source.

Parcourir les annotations sans connaître leur nombre

```
foreach ($swiss->getFeatures() as $feature) {
    printf(
        "%s %s..%s %s\n",
        $feature->getFtKey(), $feature->getFtFrom(),
        $feature->getFtTo(), $feature->getFtDesc()
    );
}

foreach ($swiss->getReferences() as $reference) {
    echo $reference->getJournal(), PHP_EOL;
}
```

3. Exemple complet - Genome

Le format GENOME reprend les anciens records statistiques DOGS. Il ne contient pas de chaîne nucléotidique : il décrit l'organisme, l'état du séquençage, les volumes GenBank et une ou plusieurs références bibliographiques. Les lignes indentées prolongent la valeur précédente.

3.2 Gb taille du génome	9.88 Gb paires de bases GB	1 234 567 entrées GB	2 références
----------------------------	-------------------------------	-------------------------	-----------------

Lire les statistiques et la bibliographie

```
$genome = DatabaseReaderFactory::readDatabase(
    'GENOME',
    file('data/sample.genome')
);

echo $genome->getOrganism(); // Homo sapiens
echo $genome->getCommonName(); // human
echo implode(' > ', $genome->getTaxClass());
echo $genome->getIsComplete(); // NO
echo $genome->getGbRelease(); // 134
echo $genome->getGbEntries(); // 1234567
echo $genome->getGbBasepairs(); // 9876543210
echo $genome->getSize(); // 3200000000

foreach ($genome->getReferences() as $ref) {
    printf("%s (%s), %s\n", $ref->getTitle(), $ref->getYear(), $ref->
getJournal());
    echo implode(' ', $ref->getAuthors());
}
```

Contrat de GenomeReference

Getter	Type	Exemple du fixture
getType()	string	journal / book
getAuthors()	string[]	Lander E.S., Linton L.M., Birren B.
getTitle()	string	Initial sequencing and analysis of the human genome.
getJournal()	string	Nature
getVolume()	string	409
getPages()	string	860-921
getYear()	string	2001

SwissProt et Genome : ce qui change

Aspect	SWISSPROT	GENOME
Identifiant d'entrée	première accession AC	nom scientifique ORGANISM
Objet principal	Sequence	ParseGenomeManager
Chaîne biologique	protéine dans SQ	aucune
Annotations	features, keywords, xrefs, auteurs, références	taxonomie, statistiques, références
Continuation	labels SwissProt répétés	ligne indentée rattachée au label précédent
Fin d'entrée	//	//

Valider la structure avant parsing

```
$lines = file('data/sample.genome');

if (!ParseGenomeManager::isEntryStart($lines[0])) {
    throw new RuntimeException('Le record ne commence pas par ORGANISM');
}

$id = ParseGenomeManager::getEntryId($lines, $lines[0]);
assert($id === 'Homo sapiens');
assert(ParseGenomeManager::isEntryEnd('/'));

```

Les labels inconnus sont ignorés. Une continuation indentée n'est assemblée que pour CLASSIFICATION, REF_AUTHOR et REF_TITLE ; conservez donc l'indentation du fichier source.

4. Lire directement ou indexer avec Doctrine

La lecture directe convient à un fichier ou à un record déjà isolé. Pour retrouver un record par identifiant dans une collection de fichiers, utilisez DatabaseManager : il persiste un index Collection / CollectionElement, puis recharge et parse l'entrée au moment du fetch.

Lecture directe

```
$formats = DatabaseParserFactory::getFormats();
$class   = DatabaseParserFactory::getParserClass('GENOME');
$parser  = DatabaseParserFactory::createParser('GENOME');
$parser->parseDataFile(file('data/sample.genome'));

// Raccourci équivalent :
$parser = DatabaseReaderFactory::readDatabase('GENOME', file('data/sample.genome'));
```

Indexation et récupération

```
use Amelaye\BioPHP\Domain\Database\Service\DatabaseManager;

$db = new DatabaseManager($entityManager, __DIR__ . '/data/');

$db->recording('proteomes', 'SWISSPROT', 'basicwiss.txt');
$swiss = $db->fetch('P01375');

$db->recording('genomes', 'GENOME', 'sample.genome');
$genome = $db->fetch('Homo sapiens');
```

Ce que fait DatabaseManager

- recording() crée ou réutilise une Collection, repère chaque début d'entrée, calcule l'identifiant et persiste un CollectionElement.
- fetch() cherche l'identifiant, vérifie le fichier, lit l'entrée puis délègue à DatabaseReaderFactory.
- fetch() renvoie false si l'identifiant est absent ; il lève une exception si le fichier n'existe pas ou si le format est inconnu.
- Le chemin de données est concaténé au nom stocké : fournissez un chemin terminé par /.

Schéma Doctrine

Dans une application Symfony, le bundle préconfigure le mapping par attributs des entités Database et Sequence. Le README propose bin/console doctrine:schema:create pour créer les tables.

5. Catalogue des 19 parseurs

Les noms de format sont sensibles à la casse : utilisez les chaînes retournées par `DatabaseParserFactory::getFormats()`. Chaque parseur fournit le même protocole statique de détection et un `parseDataFile()`, puis expose son propre modèle de résultat.

Ajouter un format

Créez une classe dans `Domain/Parser` qui implémente `ParseDatabaseInterface` (ou étend `ParseDbAbstractManager` pour le modèle `Sequence`), implémente le protocole commun, puis ajoutez sa classe à `DatabaseParserFactory::PARSERS`. C'est l'unique point d'enregistrement : les factories de lecture et d'indexation n'ont pas de switch dédié.

Protocole commun d'un parseur

Méthode	Rôle	
<code>getFormat(): string</code>	nom enregistré dans la factory	
<code>isEntryStart(string): bool</code>	reconnaît la première ligne d'une entrée	
<code>isEntryEnd(string): bool</code>	reconnaît généralement //	
<code>getEntryId(array, string): string</code>	extraît l'identifiant utilisé par l'index	
<code>parseDataFile(array)</code>	hydrate l'instance à partir des lignes du record	

Format	Classe	Données accessibles
GENBANK	<code>ParseGenbankManager</code>	modèle commun <code>Sequence</code> : séquence, features, références, auteurs, accessions
SWISSPROT	<code>ParseSwissprotManager</code>	modèle commun + keywords et références croisées <code>SpDatabank</code>
EMBL	<code>ParseEmblManager</code>	modèle commun <code>Sequence</code> , annotations et références
PDB	<code>ParsePdbManager</code>	code, classification, dépôt, titre, composés, source, technique, SEQRES, hélices, feuillets, CRYST1, atomes/HETATM
PROSITE	<code>ParsePrositeManager</code>	nom/type, accession, dates, description, pattern/matrice, résultats, commentaires, règles, xrefs
EXPASY_ENZYME	<code>ParseExpsyEnzymeManager</code>	EC id, noms, activité, cofacteurs, commentaires, maladies, xrefs PROSITE/SwissProt
PDBSTR	<code>ParsePdbstrManager</code>	member id, type moléculaire, longueur, groupe, dates
UNIGENE	<code>ParseUnigeneManager</code>	cluster, titre, expression, similarités protéiques, compte de séquences
PRINTS	<code>ParsePrintsManager</code>	nom/type, dates, description
BLOCKS	<code>ParseBlocksManager</code>	id, accession, distances min/max, description, triplet AA
AAINDEX	<code>ParseAaindexManager</code>	accession, description, auteur, titre, journal, références bibliographiques
PRODOM	<code>ParseProdomManager</code>	numéro, accession, release, nombre de domaines, noms fréquents, keywords
NCBI_LIT	<code>ParseNcbiLitManager</code>	id, titre de revue, abréviations, ISSN/eISSN, NLM id
PMD	<code>ParsePmdManager</code>	type/numéro, mutation, article, auteurs, Medline, journal, titre
HGBASE	<code>ParseHgbaseManager</code>	haplotype, allèle, bloc, population, fréquences, source, citation, soumission
PRF	<code>ParsePrfManager</code>	code/nom, source, taxonomie, journal, auteurs, titre, keywords, xrefs, séquence
PIR	<code>ParsePirManager</code>	nom/type, titre, accessions, organisme, dates, longueur, poids, checksum, keywords
EPD	<code>ParseEpdManager</code>	nom, classe, type de site, division, accessions, dates/releases, description, commentaires
GENOME	<code>ParseGenomeManager</code>	organisme, taxonomie, état, release/volumes/taille, références

6. Séquences sûres avec les value objects

Les value objects offrent une façade immuable et validée au-dessus des chaînes historiques. Ils suppriment les espaces, passent en majuscules, valident l'alphabet à la construction et renvoient une nouvelle instance pour chaque transformation.

Classe	Alphabet	Opérations spécifiques
DnaSequence	ACGTMRWSYKVHDBXN	complement(), reverseComplement(), getGcContent(), toRna()
RnaSequence	ACGUMRWSYKVHDBXN	complement(), reverseComplement(), getGcContent(), toDna()
AminoAcidSequence	ACDEFGHIKLMNPQRSTVWYX*	hasStop(), truncateAtStop(), hasUnknownResidue()

Opérations communes

Méthode	Résultat
getValue() / __toString()	chaîne normalisée
getMolType() / getAlphabet()	métadonnées de la classe
getLength() / isEmpty()	taille et vacuité
equals(other)	égalité de type et de valeur
subSequence(start, length?)	extrait immuable, index 0
reverse()	ordre inversé
countSymbol(symbol)	fréquence d'un symbole
isValid(string)	validation sans exception côté appelant

```
use Amelaye\BioPHP\Domain\Sequence\ValueObject\DnaSequence;

$dna = new DnaSequence("acgt mrws");
echo $dna;           // ACGTMRWS
echo $dna->complement(); // TGCAKYWS
echo $dna->reverseComplement();
echo $dna->getGcContent();
echo $dna->toRna();    // ACGUMRWS

if (!DnaSequence::isValid('ACGT?')) {
    // Le constructeur leverait InvalidSequenceException avec symbole et position.
}
```

MolecularSequenceFactory

fromMolType(type, chaîne) reconnaît notamment DNA, RNA, mRNA, ss-DNA, PRT, PROTEIN et AA. Si le type RNA contient pourtant T, ou si un type DNA contient U, les symboles gagnent afin de supporter la convention cDNA de GenBank/EMBL. fromEntity(Sequence) applique cette logique au résultat d'un parseur.

7. Calculs sur les séquences

SequenceBuilder est la façade stateful : injectez une entité Sequence, puis omettez la chaîne dans la plupart des appels. SequenceManager contient les calculs et accepte des chaînes explicites. Le builder est généralement le point d'entrée le plus lisible.

```
$builder->setSequence($swiss->getSequence());

$typed = $builder->getMolecularSequence();
$motifs = $builder->findPattern('C..C');
$freq = $builder->symFreq('C');
$segment = $builder->subSeq(0, 40);

// Pour une entité DNA :
$builder->setSequence($dnaEntity);
$reverseFacingStrand = strrev($builder->complement('DNA'));
$palindromes = $builder->findPalindrome(null, 0, 6);
```

Options de motifs

- I active la recherche insensible à la casse ; toute autre valeur conserve la casse.
- patPos() saute la longueur complète du motif et ne voit pas les chevauchements.
- patPoso() reprend à cutPos (1 par défaut) et voit les chevauchements.
- Les positions sont des indices PHP à partir de 0.

Dépendances de données

SequenceManager charge au constructeur les acides aminés, nucléotides et l'élément eau via les adaptateurs API. Pour un usage hors ligne, injectez vos propres implémentations des interfaces AminoApiAdapter, NucleotidApiAdapter et ElementApiAdapter.

Méthode de SequenceBuilder	Possibilité
complement(type, seq?)	complément DNA/RNA, codes IUPAC compris
halfSequence(index, seq?)	moitié gauche (0) ou droite ; ignore le pont central impair
getBridge(seq)	caractère central d'une chaîne impaire, sinon chaîne vide
expandNa(seq?)	convertit les codes dégénérés en expression régulière
molwt(limit, seq?, type?, len?)	poids DNA/RNA, limites basse/haute pour symboles ambigus
countCodons(features?, length?)	nombre de codons en tenant compte de /codon_start
subSeq(start, count, seq?)	sous-chaîne indexée à partir de 0
patPos(pattern, options, seq?)	positions sans chevauchement, groupées par motif
patPoso(pattern, options, cutPos, seq?)	positions avec chevauchement
patFreq(pattern, seq?, options)	fréquence par sous-chaîne correspondante
findPattern(pattern, seq?, options)	liste des occurrences correspondantes
symFreq(symbol, seq?)	nombre d'occurrences exactes d'un symbole
getCodon(index, seq?, frame)	codon à un index et cadre de lecture
translate(seq?, frame, format)	traduction nucléotide vers acides aminés
translateCodon(codon, format)	traduction d'un codon isolé
charge(aminoSeq)	profil de charge d'une protéine
chemicalGroup(aminoSeq)	réduction par groupes chimiques
isMirror(seq?) / findMirror(...)	détection et recherche de miroirs
isPalindrome(seq?) / findPalindrome(...)	détection et recherche de palindromes

8. Alignements et comparaison

SequenceAlignmentManager

Ce manager lit et analyse des alignements existants. Il ne calcule pas un nouvel alignement multiple. Les fixtures du dépôt illustrent FASTA et CLUSTAL.

```
$alignment->setFilename(__DIR__ . '/data/clustal.txt');
$alignment->setFormat('CLUSTAL');
$alignment->parseFile();

echo $alignment->getSeqSet()->count();
echo $alignment->getMaxLength();
echo $alignment->getGapCount();
echo $alignment->consensus(80);
```

Méthodes	Usage
setFilename() / setFormat() / parseFile()	charge un alignement ; formats FASTA et CLUSTAL
parseFasta() / parseClustal()	parse explicitement le format choisi
getSeqSet()	ArrayIterator des entités Sequence
sortAlpha(ASC DESC)	trie les séquences par identifiant
getMaxLength()	longueur maximale
getGapCount()	nombre total de gaps '-'
getIsFlush()	true si les séquences ont la même longueur
charAtRes(seqIndex, residue)	caractère à un numéro de résidu
substrBwRes(seqIndex, start, end)	segment entre résidus
colToRes() / resToCol()	conversion colonne d'alignement / résidu
subalign(start, end)	sous-alignement de colonnes
select()	sélection interne selon l'état du manager
resVar(threshold)	variabilité des résidus
consensus(threshold)	consensus, '?' sous le seuil
addSequence() / deleteSequence(id)	modifie l'ensemble, sans effectuer d'alignement

SequenceMatchManager

Méthode	Usage / limite
setSubMatrix(SubMatrix)	définit les groupes de substitutions partielles
compareLetter(a, b, matrix?, equal?, partial='+', no='.')	classe une paire de résidus
hamdist(a, b)	distance de Hamming ; longueurs obligatoirement égales
levdist(a, b, insert=1, replace=1, delete=1)	Levenshtein avec coûts ; maximum annoncé 255 symboles
xlevdist(a, b)	Levenshtein étendu, coûts fixes ; maximum 1024 symboles
match(a, b, matrix?, ...)	chaîne de correspondance exacte/partielle/non-match ; longueurs égales
partialMatch(a, b, matrix)	true si les deux symboles figurent dans une même règle

```
$matrix = new SubMatrix();
$matrix->addrule('G', 'A', 'V', 'L', 'I');
$matrix->addrule('S', 'T');

$match->setSubMatrix($matrix);
echo $match->hamdist('ACGT', 'AGGT'); // 1
echo $match->levdist('MSTES', 'MATES');
echo $match->match('GAST', 'AAST');
```

Quelle mesure choisir ?

Question	Mesure
Combien de positions différent, à longueur égale ?	Hamming
Combien d'insertions, suppressions ou remplacements ?	Levenshtein
Où sont les équivalences biochimiques ?	match() avec SubMatrix
Les chaînes dépassent 255 symboles ?	xlevdist() jusqu'à 1024, coûts fixes

Matrice de substitution

SubMatrix::addrule() reçoit un groupe de symboles considérés comme correspondances partielles. Définissez et injectez toujours la matrice avant compareLetter() ou match().

9. Protéines et enzymes de restriction

ProteinManager

Associez une entité Protein au manager. `seqLen()` renvoie le nombre de résidus ; `molwt()` renvoie [limite basse, limite haute], après soustraction de l'eau libérée entre résidus. La méthode renvoie false si un caractère sort de l'alphabet accepté.

```
$protein = new Protein();
$protein->setId('P01375');
$protein->setName('TNF');
$protein->setSequence($swiss->getSequence()->getSequence());

$proteinManager->setProtein($protein);
echo $proteinManager->seqLen();
[ $lower, $upper ] = $proteinManager->molwt();
```

RestrictionEnzymeManager

Le manager sait charger une enzyme du référentiel Type II ou enregistrer une enzyme personnalisée, rechercher des enzymes par propriété et couper la Sequence portée par un SequenceBuilder.

Méthode	Possibilité
<code>setEnzyme()</code> / <code>getEnzyme()</code>	réinitialise ou retourne l'entité Enzyme
<code>setSequenceManager(builder)</code>	branche la séquence à digérer
<code>parseEnzyme(name, pattern?, cut?, make='custom')</code>	crée une enzyme personnalisée ou charge une entrée connue
<code>cutSeq('N' 'O')</code>	fragments sans ou avec sites chevauchants
<code>getPattern(name)</code> / <code>getCutPos(name)</code> / <code>getLength(name?)</code>	métadonnées d'une enzyme
<code>findRestEn(pattern?, cutPos?, patternLen?)</code>	recherche par motif, coupure, longueur ou combinaisons supportées

```
$restriction->setSequenceManager($builder);
$restriction->parseEnzyme('EcoRI', 'GAATTC', '1', 'custom');
$fragments = $restriction->cutSeq('N');

$sameMotif = $restriction->findRestEn('GAATTC');
$cutAtOne = $restriction->findRestEn(null, 1, null);
```

Combinaisons acceptées

`findRestEn()` gère : motif seul, cutpos seul, longueur seule, motif + cutpos, cutpos + longueur. Les autres combinaisons lèvent une exception.

10. Oligonucléotides, génétique et statistiques

OligosManager

findOligos(sequence, longueur) produit la fréquence de tous les mots DNA de longueur 1 à 8, y compris les mots absents avec une valeur 0. Les méthodes findOligos2BasesLong() à findOligos8BasesLong() complètent une table partielle. findZScore() calcule les z-scores des tétranucléotides à partir des comptes 2-, 3- et 4-mères.

```
$di = $oligos->findOligos($dna, 2);  
$tri = $oligos->findOligos($dna, 3);  
$tet = $oligos->findOligos($dna, 4);  
$z = $oligos->findZScore($di, $tri, $tet);
```

GeneticsFunctions

Méthode	Résultat
CountACGT(sequence)	nombre de bases canoniques A/C/G/T
CountYRWSKMDVHB(sequence)	nombre de symboles dégénérés listés
CountCG(sequence)	nombre total de C et G
CreateInversion(sequence, complements)	complément inverse avec table fournie
RemoveNonCodingProt(sequence)	majuscule et retire les caractères hors alphabet protéique historique

MathematicsFunctions

Méthode	Règle
Mean(data)	ignore null, arrondi à 3 décimales, erreur si aucune valeur
Median(data)	trie puis renvoie la médiane
Variance(data)	variance d'échantillon (n - 1), arrondie à 3 décimales, minimum 2 valeurs

11. Adaptateurs REST et DTO

Les clients de Api/ étendent Bioapi, utilisent Guzzle + JMS Serializer et lisent les collections Hydra dans hydra:member. Chaque client possède une interface Adapter : vous pouvez substituer une source locale, un cache ou une autre API sans modifier les managers.

Client	Lecture	Données / helpers statiques
AminoApi	getAminos()	lettres, noms, poids, poids de résidu
NucleotidApi	getNucleotids()	DNA/RNA, compléments, poids
ElementApi	getElements(), getElement(id)	éléments et masse (eau utilisée par SequenceManager)
Pam250MatrixDigitApi	getPam250Matrix()	matrice PAM250
PKApi	getPkValueById(id)	valeurs pK
ProteinReductionApi	getReductions()	alphabets de réduction protéique
TmBaseStackingApi	getTmBaseStackings()	entropie et enthalpie d'empilement
TripletApi	getTriplets()	code génétique / triplets
TripletSpecieApi	getTriplets()	triplets, groupes et espèces
TypeIIEndonucleaseApi	getTypeIIEndonucleases()	enzymes Type II
TypeIIbEndonucleaseApi	getTypeIIbEndonucleases()	enzymes Type IIb
TypeIIIsEndonucleaseApi	getTypeIIIsEndonucleases()	enzymes Type IIIs
VendorApi	getVendors()	fournisseurs
VendorLinkApi	getVendorLinks()	liens fournisseur / enzyme

Construction standalone

```
use GuzzleHttpClient;
use JMS\Serializer\SerializerBuilder;
use Amelaye\BioPHP\Api\{AminoApi, NucleotidApi, ElementApi};
use Amelaye\BioPHP\Domain\Sequence\Service\SequenceManager;

$client = new Client(['base_uri' => 'https://api.amelayes-biophp.net']);
$serializer = SerializerBuilder::create()->build();

$manager = new SequenceManager(
    new AminoApi($client, $serializer),
    new NucleotidApi($client, $serializer),
    new ElementApi($client, $serializer)
);
```

Clé API

Le constructeur Bioapi accepte un troisième argument \$apiKey. Selon votre déploiement, transmettez cette valeur lors de la construction des clients ou encapsulez l'authentification dans votre propre adaptateur.

HTTP configuré dans le bundle

Le fichier services.xml du snapshot définit actuellement <http://api.amelayes-biophp.net>, tandis que le README montre <https://api.amelayes-biophp.net>. Pour la confidentialité et l'intégrité, redéfinissez le client Guzzle en HTTPS dans votre application.

12. Intégration Symfony

Le bundle charge trois fichiers de services XML (Database, Api, Sequence), ajoute les mappings Doctrine par attributs et fournit des alias d'interfaces. Les services sont privés : préférez l'injection de type par interface dans vos contrôleurs et services applicatifs.

```
// config/bundles.php
return [
    // ...
    Amelaye\BioPHP\AmelayeBioPHPBundle::class => ['all' => true],
];

// Puis, si vous utilisez l'index Doctrine :
// bin/console doctrine:schema:create
```

Services principaux

ID de service	Classe / alias
amelaye_biophp.database_manager	DatabaseManager
amelaye_biophp.seq_sequences	SequenceManager
amelaye_biophp.seq_sequences_bld	SequenceBuilder / SequenceInterface
amelaye_biophp.seq_alignment	SequenceAlignmentManager / SequenceAlignmentInterface
amelaye_biophp.seq_match	SequenceMatchManager / SequenceMatchInterface
amelaye_biophp.seq_proteins	ProteinManager / ProteinInterface
amelaye_biophp.seq_restenzyme	RestrictionEnzymeManager / RestrictionEnzymeInterface
amelaye_biophp.seq_oligos	OligosManager / OligosInterface
amelaye_biophp.api_amino / api_nucleotid / api_element	adaptateurs de base pour SequenceManager
amelaye_biophp.api_matrix / api_pk / api_triplet	référentiels supplémentaires
amelaye_biophp.api_t2b / api_t2s / api_ts	endonucleases
amelaye_biophp.api_vendor / api_vendorlink	fournisseurs

Paramètre de fichiers

Le paramètre XML files.path vaut ./data/. Redéfinissez-le si vos collections sont ailleurs. Le nœud de configuration amelaye_biophp existe mais n'offre pas encore d'options publiques dans ce snapshot.

Injection recommandée

```
use Amelaye\BioPHP\Domain\Sequence\Interfaces\SequenceInterface;

final class ProteinInspector
{
    public function __construct(private SequenceInterface $sequences) {}

    public function inspect(Sequence $sequence): array
    {
        $this->sequences->setSequence($sequence);
        return $this->sequences->findPattern('C..C');
    }
}
```

13. Recettes de bout en bout

SwissProt vers résumé JSON

```
$p = DatabaseReaderFactory::readDatabase('SWISSPROT', file('data/basicswiss.txt'));
$s = $p->getSequence();

$payload = [
    'accession' => $s->getPrimAcc(),
    'description'=> $s->getDescription(),
    'length' => $s->getSeqLength(),
    'taxonomy' => $s->getOrganism(),
    'featureCount' => count($p->getFeatures()),
    'referenceCount' => count($p->getReferences()),
];

echo json_encode($payload, JSON_PRETTY_PRINT | JSON_THROW_ON_ERROR);
```

Genome vers fiche de suivi

```
$g = DatabaseReaderFactory::readDatabase('GENOME', file('data/sample.genome'));

$coverageRatio = $g->getSize() > 0
    ? $g->getGbBasepairs() / $g->getSize()
    : null;

printf(
    "%s | complete=%s | release=%s | ratio=%.2f\n",
    $g->getOrganism(),
    $g->getIsComplete(),
    $g->getGbRelease(),
    $coverageRatio
);
```

Traiter plusieurs formats uniformément

```
foreach (DatabaseParserFactory::getFormats() as $format) {
    printf("%-16s %s\n", $format, DatabaseParserFactory::getParserClass($format));
}

function parse(string $format, string $path): ParseDatabaseInterface
{
    $lines = file($path, FILE_IGNORE_NEW_LINES);
    if ($lines === false) {
        throw new RuntimeException("Lecture impossible: $path");
    }
    return DatabaseReaderFactory::readDatabase($format, $lines);
}
```

Envelopper immédiatement les séquences parsées

```
$record = DatabaseReaderFactory::readDatabase('SWISSPROT', file($path));

try {
    $typed = MolecularSequenceFactory::fromEntity($record->getSequence());
} catch (InvalidSequenceException $e) {
    // Le message indique le symbole invalide et sa position.
    throw new DomainException('Record biologique invalide', 0, $e);
}
```

14. Erreurs, limites et bonnes pratiques

Situation	Comportement	Conseil
Format inconnu	exception listant les formats valides	valider avec getFormats()
Identifiant absent dans l'index	DatabaseManager::fetch() renvoie false	tester explicitement === false
Fichier indexé absent	FileNotFoundException enveloppée	chemin absolu ou paramètre files.path maîtrisé
Symbole moléculaire invalide	InvalidSequenceException	construire un value object au bord du système
Appel API indisponible	échec au constructeur de certains managers	injecter un adaptateur local/cache
Hamming / match de tailles différentes	exception	aligner ou tronquer explicitement avant comparaison
Levenshtein trop long	limites 255 / 1024 annoncées	segmenter ou employer un algorithme dédié
Alignement non flush	getIsFlush() false	ne pas supposer des colonnes homologues
Oligos hors 1..8	entrée non supportée	valider la longueur avant appel
Version alpha	API susceptible d'évoluer	verrouiller la version Composer et exécuter les tests

Checklist de production

- Verrouiller une version ou un commit précis dans Composer.
- Conserver les fichiers sources biologiques et leur provenance avec l'index Doctrine.
- Valider les alphabets au moyen des value objects dès l'entrée du système.
- Définir la convention de coordonnées (0-based API vs annotations biologiques) dans votre application.
- Remplacer ou mettre en cache les adaptateurs REST pour les traitements reproductibles.
- Tester vos propres fixtures : le parsing dépend des espaces, labels et continuations du format source.
- Utiliser HTTPS pour le client de données biologiques.

Vérification du snapshot documenté

La suite complète passe sous PHP 8.5.9 : 405 tests et 1 198 assertions. Les tests ciblés ParseSwissprotManagerTest et ParseGenomeManagerTest représentent 8 tests et 53 assertions. L'avertissement de couverture indique seulement qu'aucun pilote de couverture n'est installé dans l'environnement de génération.

Point de départ conseillé

Commencez avec DatabaseReaderFactory et MolecularSequenceFactory. Ajoutez DatabaseManager lorsque vous avez besoin d'un index persistant, puis SequenceBuilder pour les calculs.

Annexe A. Référence rapide

Imports essentiels

```
use Amelaye\BioPHP\Domain\Database\Factory\DatabaseParserFactory;
use Amelaye\BioPHP\Domain\Database\Factory\DatabaseReaderFactory;
use Amelaye\BioPHP\Domain\Database\Service\DatabaseManager;
use Amelaye\BioPHP\Domain\Sequence\Builder\SequenceBuilder;
use Amelaye\BioPHP\Domain\Sequence\Entity\Sequence;
use Amelaye\BioPHP\Domain\Sequence\ValueObject\DnaSequence;
use Amelaye\BioPHP\Domain\Sequence\ValueObject\RnaSequence;
use Amelaye\BioPHP\Domain\Sequence\ValueObject\AminoAcidSequence;
use Amelaye\BioPHP\Domain\Sequence\ValueObject\MolecularSequenceFactory;
```

Formats acceptés

GENBANK, SWISSPROT, EMBL, PDB, PROSITE, EXPASY_ENZYME, PDBSTR, UNIGENE, PRINTS, BLOCKS, AAINDEX, PRODOM, NCBI_LIT, PMD, HGBASE, PRF, PIR, EPD, GENOME.

Choisir le bon point d'entrée

Besoin	Point d'entrée
Parser un fichier immédiatement	DatabaseReaderFactory::readDatabase()
Découvrir les formats	DatabaseParserFactory::getFormats()
Indexer et retrouver par id	DatabaseManager::recording() / fetch()
Valider et transformer une chaîne	DnaSequence / RnaSequence / AminoAcidSequence
Transformer un résultat parsé	MolecularSequenceFactory::fromEntity()
Analyser une entité Sequence	SequenceBuilder
Lire un alignement	SequenceAlignmentManager
Comparer deux chaînes	SequenceMatchManager
Poids d'une protéine	ProteinManager
Digestion de restriction	RestrictionEnzymeManager
Fréquences de k-mères	OligosManager
Données de référence distantes	classes Api + interfaces Adapter

Fin du guide

Les exemples de ce document sont alignés sur les fixtures et l'API publique du dépôt au 25 août 2026.